

1. Employee sequence by salary - ROW_NUMBER() Write a query to display emp_name, department, salary, and a unique row number for all employees ordered by salary from highest to lowest. Use ROW_NUMBER().

ANS: select (row_number() over (order by salary desc)) as s_no, emp_name, department, salary from employees;

2. Salary rank across the company - RANK() Write a query to rank all employees by salary in descending order. Employees with the same salary must receive the same rank, and the next rank should be skipped.

ANS: select (rank() over (order by salary desc)) as rank_no, emp_name, department, salary from employees;

3. Salary rank without gaps - DENSE_RANK() Write a query to rank all employees by salary in descending order, but do not leave gaps in the ranking when two or more employees have the same salary.

ANS: select (dense_rank() over (order by salary desc)) as s_no, emp_name, department, salary from employees;

4. Row number inside each department - ROW_NUMBER() Write a query to assign a row number to employees separately inside each department. Within every department, the highest-paid employee must receive row number 1.

ANS: select (row_number() over (partition by department order by salary desc)) as s_no, emp_name, department, salary from employees;

5. Department-wise salary rank - RANK() Write a query to rank employees by salary within each department. If two employees in the same department have the same salary, they must share the same rank.

ANS: select (rank() over (partition by department order by salary desc)) as rank_no, emp_name, department, salary from employees;

6. Top 2 distinct salaries in each department - DENSE_RANK() Write a query to return only employees who belong to the top 2 distinct salary levels in their department. Use DENSE_RANK() in a subquery or CTE.

ANS: select x.rank_no,x.emp_id,x.emp_name,x.department from (select (dense_rank() over (partition by department order by salary desc)) as rank_no,emp_id,emp_name,department from employees) as x where x.rank_no<=2;

7. Highest-paid employee name per department - FIRST_VALUE() Write a query that displays every employee along with the name of the highest-paid employee in that employee's department. Use FIRST_VALUE().

ANS: select (first_value(emp_name) over (partition by department order by salary desc)) as e_name, emp_name,department,salary from employees;

8. Lowest-paid employee name per department - LAST_VALUE() Write a query that displays every employee along with the name of the lowest-paid employee in that department. Use LAST_VALUE() with an explicit window frame so the result is correct for all rows.

ANS: select emp_name,department,salary, (last_value(emp_name) over (partition by department order by salary)) as e_name from employees;

9. Compare first and last salary in each department Write a query that displays emp_name, department, salary, highest salary in the department, and lowest salary in the department using FIRST_VALUE() and LAST_VALUE(). Order the window by salary DESC and use the correct frame.

ANS: select emp_name, department,salary, (first_value(salary) over (partition by department order by salary desc)) as highest,(last_value(salary) over (partition by department order by salary desc)) as lowest from employees;

10. Find the 2nd highest distinct salary employees - DENSE_RANK() Write a query to return all employees whose salary is the second-highest distinct salary in the entire company. Do not use LIMIT. Use DENSE_RANK().

ANS: select x.emp_name,x.salary from (select (dense_rank() over(order by salary desc)) as rank_no,emp_name,salary from employees) as x where x.rank_no=2;

DSA:

1. Delete a node from a binary search tree.

```
2. class Solution {
3.     TreeNode findSuccessor(TreeNode root) {
4.         while (root.left != null) {
5.             root = root.left;
6.         }return root;
7.     }
8.     public TreeNode deleteNode(TreeNode root, int key) {
9.         if (root == null) return null;
10.        if (key > root.val) {
11.            root.right = deleteNode(root.right, key);
12.        } else if (key < root.val) {
13.            root.left = deleteNode(root.left, key);
14.        } else {
15.            if (root.left == null) return root.right;
16.            if (root.right == null) return root.left;
17.            TreeNode temp = findSuccessor(root.right);
18.            root.val = temp.val;
19.            root.right = deleteNode(root.right, temp.val);
20.        }return root;
21.    }
22. }
```

2. Check if a binary tree is a binary search tree.

```
class Solution {
    boolean check(TreeNode root, long min, long max) {
        if (root == null) return true;
        if (root.val <= min || root.val >= max) {
            return false;
        }return check(root.left, min, root.val) &&
            check(root.right, root.val, max);
    }boolean isBST(TreeNode root) {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
}
```

3. Find the kth smallest and kth largest elements in a BST.

```
class Solution {
    List<Integer> result = new ArrayList<>();
    void inorder(TreeNode root){
        if(root==null) return;
        inorder(root.left);
        result.add(root.val);
        inorder(root.right);
    }
    public int[] kthSmallestAndLargest(TreeNode root, int k) {
```

```
        inorder(root);  
        return int[]{result.get(k-1),result.get(result.length()-k)};  
    }  
}
```

4. Find whether the given BST is balanced or not.

```
class Solution {  
    boolean flag = true;  
    int height(TreeNode root){  
        if(root == null) return 0;  
        int lh = height(root.left);  
        int rh = height(root.right);  
        if(Math.abs(lh-rh)>1) flag = false;  
        return 1+Math.max(lh,rh);  
    }  
    public boolean isBalanced(TreeNode root) {  
        height(root);  
        return flag;  
    }  
}
```